

LECTURE 03

Flutter Basics

Widgets, the building blocks of every Flutter app

Instructor · Muhammad Sabah

What we'll cover today

01

Everything is a Widget

The Widget Tree, MaterialApp, and Scaffold

02

Stateless vs Stateful

Two kinds of widgets and when to use each

03

Essential Widgets

Text, Image, Icon, Container, Buttons, AppBar

04

Hot Reload

See your changes instantly while you build



PART ONE

Everything is a Widget

The one big idea behind Flutter

What is a Widget?

A Widget is a description of part of the user interface.

- **Visible** Buttons, text, images, icons
- **Layout** Rows, columns, padding, alignment
- **Logical** Themes, gestures, navigation

REMEMBER

In Flutter,
everything
is a Widget.

A button, a screen, padding, even your whole app — all widgets.

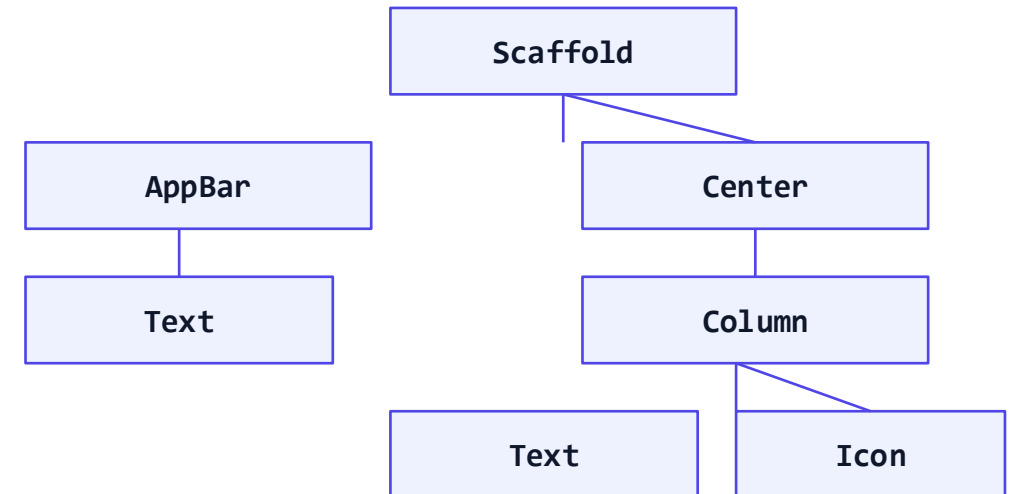
The Widget Tree

Widgets are nested inside other widgets to form a tree.

CODE

```
Scaffold(  
  appBar: AppBar(  
    title: Text('My App'),  
  ),  
  body: Center(  
    child: Column(  
      children: [  
        Text('Hello'),  
        Icon(Icons.star),  
      ],  
    ),  
  ),  
)
```

AS A TREE



Anatomy of main.dart

Every Flutter app starts here.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Hello!'),
        ),
      ),
    );
  }
}
```

WHAT'S HAPPENING

import

Brings in Flutter's Material Design library

main()

The Dart entry point of your app

runApp()

Tells Flutter to draw your widget on screen

build()

Returns the widget tree to display

MaterialApp & Scaffold

These two widgets give your app its skeleton.

THE APP WRAPPER

MaterialApp

Sets up the whole app — themes, navigation, and Material Design defaults.

- Wraps your entire app
- Defines the theme (colors, fonts)
- Manages routes/navigation
- Used once, at the root

THE PAGE LAYOUT

Scaffold

Provides the basic visual structure of a single screen.

- AppBar at the top
- Body in the middle
- FloatingActionButton
- BottomNavigationBar, Drawer

PART TWO

Stateless vs Stateful

Two flavors of widget — pick the right one

StatelessWidget

A widget that never changes after it's built.

EXAMPLE

```
class WelcomeCard extends StatelessWidget {  
  final String name;  
  
  WelcomeCard({required this.name});  
  
  @override  
  Widget build(BuildContext context) {  
    return Text('Welcome, $name!');  
  }  
}  
  
// Use it:  
WelcomeCard(name: 'Hama')
```

USE WHEN

- ✓ **Static text**
A welcome message, a title
- ✓ **Display-only data**
Profile picture, app logo
- ✓ **Layout containers**
A reusable card design

Once built, the data inside cannot change.

StatefulWidget

A widget that can change over time.

EXAMPLE

```
class Counter extends StatefulWidget {  
  @override  
  State<Counter> createState() => _CounterState();  
}  
  
class _CounterState extends State<Counter> {  
  int count = 0;  
  
  @override  
  Widget build(BuildContext context) {  
    return Text('Count: $count');  
  }  
}
```

USE WHEN

✓ User input

Forms, text fields, switches

✓ Counters

Anything that increments/changes

✓ Loading data

Showing fetched data on screen

Has a separate State class that holds the data.

setState — making it update

Call `setState()` to tell Flutter the data changed and the UI should rebuild.

```
class _CounterState extends State<Counter> {
  int count = 0;

  void _increment() {
    setState(() {
      count++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Count: $count'),
        ElevatedButton(
          onPressed: _increment,
          child: Text('Add 1'),
        ),
      ],
    );
  }
}
```

THE RULE

Change data inside `setState()`

If you change a variable without `setState`, the UI won't update — the data changes silently.

`count++` → no rebuild

`setState(() => count++)` → rebuilds



PART THREE

Essential Widgets

Your day-one toolkit

Text

Display a string of text on the screen.

EXAMPLE

```
Text('Hello, Flutter!')

// With styling
Text(
  'Welcome',
  style: TextStyle(
    fontSize: 24,
    fontWeight: FontWeight.bold,
    color: Colors.indigo,
  ),
)
```

TEXTSTYLE PROPERTIES

fontSize

Size in logical pixels (e.g. 16, 24)

fontWeight

FontWeight.normal, .bold, w400, w700

color

Colors.red, Colors.indigo, Color(0xFF...)

fontStyle

FontStyle.italic

letterSpacing

Space between characters

Image & Icon

Show pictures and built-in icons.

IMAGE

Image

```
// From the web
Image.network(
  'https://example.com/cat.png',
)

// From assets folder
Image.asset('images/logo.png')
```

ICON

Icon

```
Icon(
  Icons.favorite,
  size: 32,
  color: Colors.red,
)

// 1000+ icons in Icons.*
```

Container — the box

A box you can size, color, pad, and decorate. The Swiss-army widget.

```
Container(  
  width: 200,  
  height: 100,  
  padding: EdgeInsets.all(16),  
  margin: EdgeInsets.all(8),  
  decoration: BoxDecoration(  
    color: Colors.indigo,  
    borderRadius: BorderRadius.circular(12),  
  ),  
  child: Text(  
    'Hello',  
    style: TextStyle(color: Colors.white),  
  ),  
)
```

WHAT YOU CAN SET

width / height	Fixed size in pixels
padding	Space inside the container
margin	Space outside the container
color	Background color
decoration	Border, radius, shadow, gradient
child	What goes inside

Buttons

Three flavors for three different moments in your UI.

ELEVATEDBUTTON

ElevatedButton

Filled, prominent — for the main action.

```
ElevatedButton(  
  onPressed: () {  
    print('Saved!');  
  },  
  child: Text('Save'),  
)
```

TEXTBUTTON

TextButton

Flat — for secondary actions.

```
TextButton(  
  onPressed: () {  
    print('Cancel');  
  },  
  child: Text('Cancel'),  
)
```

ICONBUTTON

IconButton

Just an icon — for compact actions.

```
IconButton(  
  onPressed: () {},  
  icon: Icon(  
    Icons.favorite,  
  ),  
)
```

Every button needs an onPressed callback. Pass null to disable it.

AppBar

The bar at the top of your screen — title, actions, back button.

INSIDE A SCAFFOLD

```
Scaffold(  
  appBar: AppBar(  
    title: Text('My App'),  
    backgroundColor: Colors.indigo,  
    actions: [  
      IconButton(  
        icon: Icon(Icons.search),  
        onPressed: () {},  
      ),  
      IconButton(  
        icon: Icon(Icons.more_vert),  
        onPressed: () {},  
      ),  
    ],  
  ),  
  body: Center(child: Text('Content')),  
)
```

COMMON PROPERTIES

title	The Text widget shown in the bar
backgroundColor	Bar color
leading	Widget on the far left
actions	Icon buttons on the right
elevation	Shadow depth (0 = flat)
centerTitle	true to center the title

Hot Reload — your superpower

Change your code, save, and see it live in under a second.



press

Hot Reload

Updates UI, keeps app state. Use this 90% of the time.



press

Hot Restart

Resets state and rebuilds the app. Use after big changes.



press

Quit

Stop the running app.

Tip — In VS Code and Android Studio, just save the file (Ctrl/Cmd + S) and Hot Reload runs automatically.

W R A P U P

What we covered today

Y O U N O W K N O W

- ✓ Everything in Flutter is a widget
- ✓ How widgets nest into a tree
- ✓ MaterialApp & Scaffold structure
- ✓ Stateless vs Stateful widgets
- ✓ Updating UI with setState
- ✓ Text, Image, Icon, Container
- ✓ Buttons and AppBar
- ✓ Hot reload workflow

N E X T L E C T U R E

- Layouts — Row & Column
- Spacing with Padding & SizedBox
- Alignment & MainAxis / CrossAxis
- Expanded & Flexible
- ListView for scrolling content
- Building real, multi-widget screens
- Mini project — a profile card UI



THANK YOU

Questions?